CSS4004 — DISTRIBUTED SYSTEMS

CONSISTENCY MODELS & THE CAP THEOREM

Immutability, PACELC Trade-offs, and Quorum Tuning in Distributed Key-Value Stores

---

Laboratory Report Student:

Miras Zhanbyr

Group: 4

Environment: Windows 10, Python 3.10.9, gRPC, pytest

Abstract

This laboratory implements and evaluates a three-replica distributed key-value store using Python and gRPC. The system demonstrates two read modes—linearizable-style reads through a designated leaseholder and bounded-staleness reads served locally by followers—and two write strategies: majority-based CP writes and availability-oriented AP writes. Network partitions are simulated between a two-node majority group and a one-node minority group. The experiments examine quorum intersection, Last-Write-Wins conflict resolution, idempotent retries, follower staleness limits, and read latency. The automated test suite reports 10 passing tests. In the healthy-cluster benchmark, follower reads achieved a median latency of 1.74 ms compared with 4.31 ms for linearizable reads, while both modes recorded 100.0% read availability over 1,000 requests per scenario.

Contents

Section	Title
1	Introduction and objectives
2	System architecture and environment
3	Protocol and data model
4	Implementation of consistency strategies
5	CAP partition experiments
6	Quorum verification
7	Automated test results
8	PACELC performance evaluation
9	Discussion and implementation limitations
10	Conclusion
Appendix	Reproduction commands and references

1. Introduction and Objectives

Distributed key-value stores keep data on multiple nodes so that services can continue operating when individual machines or network links fail. Replication improves fault tolerance and can increase read capacity, but it also introduces consistency challenges: replicas may temporarily contain different values, and a client may not be able to contact every node.

The purpose of this laboratory is to extend a Python/gRPC replicated service and observe the trade-offs described by the CAP theorem and the PACELC model. The implementation uses three replica processes. Replica A is configured as the leaseholder, while replicas B and C act as followers.

The practical objectives are:

• implement configurable read consistency and write strategies;
• require a majority for CP writes and permit a single available replica to accept AP writes;
• simulate a partition between nodes A+B and node C;
• reconcile divergent values using Last-Write-Wins (LWW);
• verify quorum intersection using R + W > N;
• measure median latency, p95 latency, and read availability for two read paths.

2. System Architecture and Environment

The service consists of three gRPC replicas and a client. Each replica maintains an in-memory key-value map. A stored item is represented as a timestamp-value pair: (timestamp_ms, value). The timestamp is used to compare versions during replication and conflict resolution.

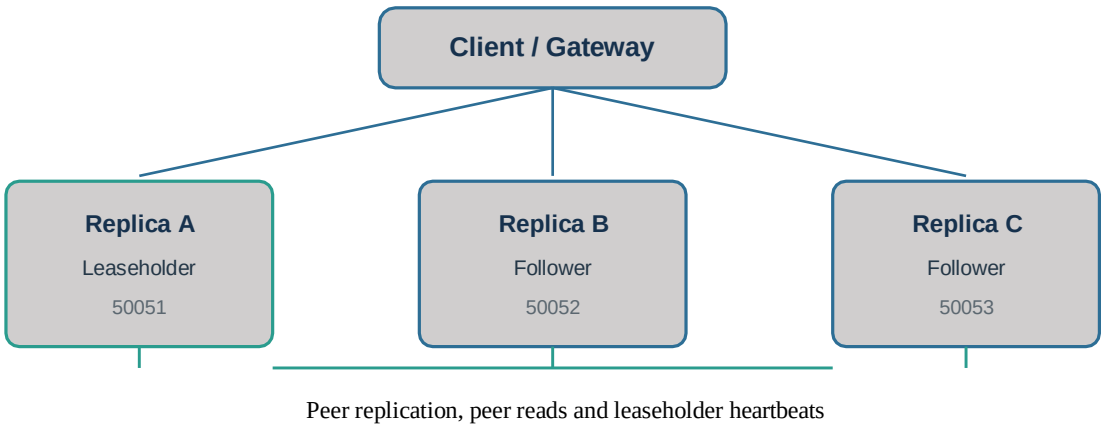


Figure 1. Three-replica architecture. The client can connect to any replica. The leaseholder coordinates linearizable-style reads and sends periodic heartbeats to followers. Replicas communicate using internal gRPC methods.

Component	Role	Address / port
Replica A	Leaseholder for the configured keyspace	127.0.0.1:50051
Replica B	Follower	127.0.0.1:50052
Replica C	Follower; benchmark client attaches here	127.0.0.1:50053
Client	Issues Get and Put requests	Connects to a replica

The benchmark runs three replica processes on localhost. The recorded environment is Windows 10, Python 3.10.9, and 12 logical CPUs. The automated partition tests create three in-process gRPC servers on ephemeral localhost ports and use a peer-blocking mechanism to simulate lost inter-replica links.

3. Protocol and Data Model

The Protocol Buffers interface defines the public Get and Put operations and three internal RPCs used for replica coordination. The public messages carry the requested consistency mode, maximum staleness, write strategy, timestamp, and idempotency key.

Protocol element	Purpose
ReadConsistency.LINEARIZABLE	Requests the leaseholder-based majority read path.
ReadConsistency.FOLLOWER_STALE	Allows a replica to answer from its local state, subject to the staleness limit.
WriteStrategy.QUORUM_CP	Requires acknowledgements from a majority of replicas.
WriteStrategy.ALWAYS_AP	Accepts a write at the receiving replica even if peers are unreachable.
GetReply.is_stale	Indicates whether the response came from a follower.
PutReply.acks_received	Reports the number of replicas that acknowledged the write, including the coordinator.
idempotency_key	Identifies a logical write so a retry does not create a second operation.

The internal RPCs are Replicate (copy a write to a peer), PeerRead (read a peer's local value), and Heartbeat (allow followers to track recent contact with the leaseholder). These RPCs extend the assignment's public Get/Put interface to support the experiments.

4. Implementation of Consistency Strategies

4.1 Write path: CP and AP

The replica receiving a Put request acts as the coordinator. It applies the value locally and sends a Replicate request to its configured peers. The acknowledgement count includes the local replica plus successful peer acknowledgements. For N = 3, the majority requirement is floor(N/2) + 1 = 2.

Strategy	Acceptance rule	Observed behaviour
QUORUM_CP	Success only when at least 2 of 3 replicas acknowledge.	A write on the A+B side succeeds with 2 acknowledgements; a write on isolated C fails with 1 acknowledgement.
ALWAYS_AP	The receiving replica accepts the write locally, even without peer acknowledgements.	A write on isolated C succeeds with 1 acknowledgement. Divergent values may exist until recovery.

4.2 Linearizable-style read path

A request using **LINEARIZABLE** is forwarded to the configured leaseholder if it first reaches a follower. The leaseholder reads its own local value and requests local values from peers. It requires a majority of responses (R = 2), selects the highest timestamp-value pair, and performs read repair by updating replicas that returned an older value. If fewer than two replicas respond, the request fails with **UNAVAILABLE**.

This is a majority-based linearizable-style read implementation for the lab. It demonstrates leader routing, quorum reads, version comparison, and read repair. It should not be interpreted as a production-grade formal linearizability protocol: the project does not implement a consensus algorithm, lease fencing, or a durable replicated log, and writes may be coordinated by any receiving replica.

4.3 Bounded-staleness follower read path

A request using **FOLLOWER_STALE** is answered directly from the receiving replica's local in-memory state. The leaseholder sends heartbeats every 0.5 seconds. A follower estimates the age of its last heartbeat. If max_staleness_ms is non-zero and the heartbeat age exceeds the configured limit, the server returns **FAILED_PRECONDITION** with the message **STALE_READ_EXCEEDED.** Otherwise, it returns the local value and marks is_stale = true for a follower response.

The heartbeat check is a liveness-based approximation of staleness. It confirms recent contact with the leaseholder, but it does not prove that every write has been replicated. This limitation is considered in Section 9.

1.2 Conflict resolution and idempotency

LWW compares stored (timestamp, value) pairs and keeps the larger pair. The timestamp is the primary ordering field; the value provides a deterministic tie-break if timestamps are equal. When the partition heals, a majority read can discover the newer version and read repair propagates it to replicas that answered with older data. The idempotency map stores the timestamp associated with each idempotency key so that retrying the same logical operation reuses its original timestamp.

1. CAP Network Partition Experiments

The test suite simulates a network partition that separates the cluster into two groups: DC-1 contains replicas A and B (two of three nodes), while DC-2 contains replica C (one of three nodes). The nodes remain reachable by the test client, but peer-to-peer calls across the partition are blocked.

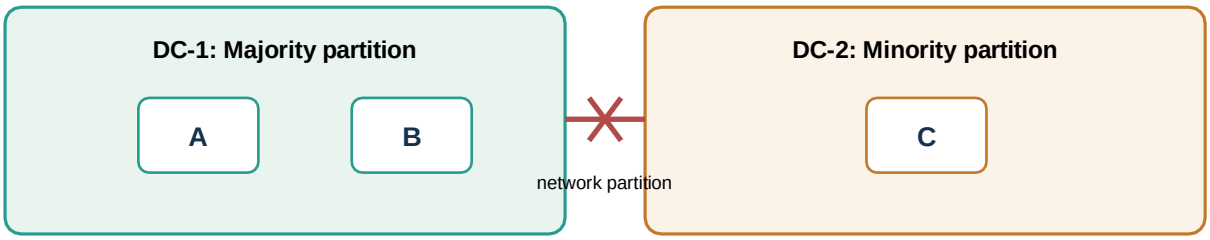


Figure 2. Partition layout used in the automated tests. DC-1 retains a majority; DC-2 contains only the isolated minority replica.

1.1 CP behaviour

On the majority side, a QUORUM_CP write sent to A or B can receive the local acknowledgement plus one peer acknowledgement. The operation succeeds with 2 acknowledgements. On the minority side, C can acknowledge only its local copy, so the operation returns success = false and reports quorum loss.

Test case	Available replicas	Expected / observed result
CP write to DC-1 (A+B)	2 of 3	Success; acks_received = 2
CP write to DC-2 (C)	1 of 3	Failure; acks_received = 1; quorum lost

1.2 AP behaviour and recovery

With ALWAYS_AP, replica C accepts a write while isolated and reports success with one acknowledgement. This keeps the write path available, but the partition can create divergent values. After the links are restored, a linearizable-style majority read selects the highest timestamp and read repair propagates the winning value. The tests cover both cases: the DC-2 value is newer, and the DC-1 value is newer. A separate test confirms that an older timestamp arriving later does not overwrite a newer value.

The assignment mentions Union Merge as another conflict-resolution approach. The submitted implementation uses timestamp-based LWW; it does not implement a general-purpose union merge. Union merge is suitable only for data types with a well-defined merge operation, such as sets, and is not used by this key-value store.

2. Quorum Verification

Let N be the total number of replicas, R the number of replicas participating in a read, and W the number acknowledging a write. A read quorum is guaranteed to intersect a write quorum when R + W > N, assuming the write is stored on all W acknowledging replicas and the read selects the newest version among the replicas it contacts.

N	R	W	R + W	Intersection condition
3	2	2	4	Satisfied: 4 > 3
3	1	2	3	Not satisfied: 3 is not greater than 3
3	2	1	3	Not satisfied: 3 is not greater than 3
3	1	1	2	Not satisfied: 2 is not greater than 3

In the implemented configuration, a CP write requires W = 2 and the leaseholder's majority read uses R = 2. Therefore R + W = 4, which is greater than N = 3. In the partition test, the successful write is stored on A and B. Every possible two-replica read set contains at least one of those two replicas, while a one-replica read from C can miss the write. The automated test checks these combinations.

The intersection rule is a necessary quorum property, not a complete consistency protocol by itself. Correct version ordering, membership, failure handling, and a safe write protocol are also required for production guarantees.

3. Automated Test Results

The command python -m pytest tests/test_cap.py -vcollected and executed 10 tests. All 10 passed in 2.29 seconds on Windows with Python 3.10.9 and pytest 9.2.0.

#	Test	Purpose / verified behaviour	Result
1	test_cp_put_succeeds_on_majority_side	CP write succeeds on A+B with 2 acknowledgements.	PASS
2	test_cp_put_fails_on_minority_side	CP write fails on isolated C with 1 acknowledgement.	PASS
3	test_ap_put_accepted_on_minority_side	AP write is accepted on C with 1 acknowledgement.	PASS
4	test_lww_merges_divergent_values_after_heal (DC-2 newer)	Newer timestamp from DC-2 wins after recovery.	PASS
5	test_lww_merges_divergent_values_after_heal (DC-1 newer)	Newer timestamp from DC-1 wins after recovery.	PASS
6	test_lww_ignores_arrival_order	Older value arriving later does not replace newer value.	PASS
7	test_read_quorum_intersects_write_quorum	For N=3, R=2, W=2, every read quorum intersects the write quorum.	PASS
8	test_follower_read_rejected_when_too_stale	Follower read is rejected after heartbeat age exceeds 1000 ms.	PASS
9	test_linearizable_read_refused_when_leaseholder_isolated	Majority read fails when the isolated leaseholder cannot obtain a quorum.	PASS
10	test_retry_after_failed_quorum_applies_once	Retry with the same idempotency key reuses the original timestamp.	PASS

Test summary	10 passed	0 failed	2.29 s

The tests run replicas in-process on ephemeral localhost ports. Partitioning is injected by configuring each replica to block peer addresses outside its assigned group. The benchmark, in contrast, starts three separate operating-system processes.

5. PACELC Performance Evaluation

The benchmark compares two read paths while all three replicas are healthy. The client is connected to replica C. For the linearizable scenario, C forwards each request to leaseholder A, which performs a majority read. For the follower-stale scenario, C answers from its local state. The script performs 50 warm-up reads for each mode and then records 1,000 sequential reads per scenario.

Read mode	Target node	Median latency (ms)	p95 latency (ms)	Read availability
Linearizable	Leaseholder (remote path)	4.31	6.59	100.0%
Follower Stale	Local follower	1.74	2.27	100.0%

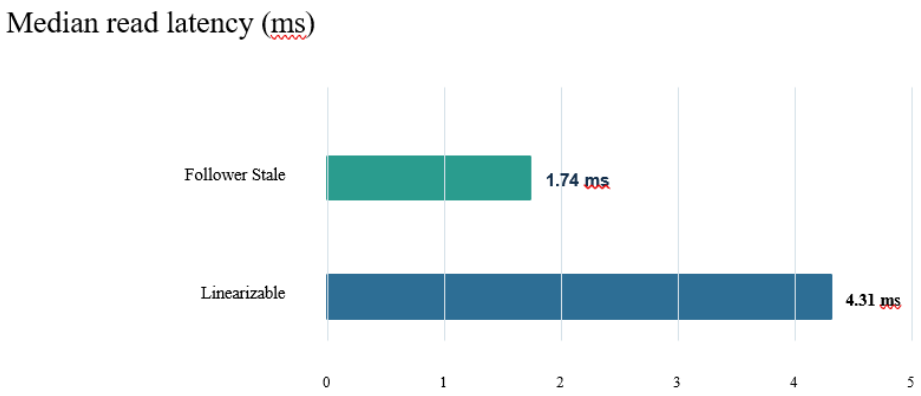


Figure 3. Median read latency measured by the submitted benchmark. Lower latency indicates a faster response path.

The follower-stale path achieved a median latency of 1.74 ms, compared with 4.31 ms for the linearizable path. This is approximately 2.48 times faster by median latency, or a 59.6% reduction. Its p95 latency was 2.27 ms compared with 6.59 ms, approximately 65.6% lower. Both scenarios completed all 1,000 reads successfully, giving 100.0% measured availability in the healthy-cluster run.

The results are consistent with the expected PACELC trade-off in the ELSE case: when there is no partition, a local follower read can reduce latency by avoiding a remote request to the leaseholder and its peer-read round. The benchmark is a localhost experiment; its absolute latency values should not be treated as a prediction of performance across a real network or data centre.

6. Discussion and Implementation Limitations

The experiments demonstrate the main teaching goals: majority-based CP writes reject operations on the minority side; AP writes remain available on an isolated replica; divergent versions can be reconciled using LWW; and a follower read can be faster than a leaseholder-based majority read in a healthy local cluster. The test suite also verifies the quorum-intersection arithmetic and bounded-staleness rejection.

The implementation is a teaching prototype and has several limitations that are important when interpreting the results:

1. **In-memory storage.** Replica state is not persisted. A restarted process starts with an empty map, so the implementation does not provide crash-safe durability.
2. **Simulated partition.** Tests block peer RPCs inside the service. They do not use a TCP proxy, firewall rule, or real network disconnection.
3. **Failed CP writes are not rolled back.** The coordinator applies the value locally before it knows whether a majority will acknowledge it. A failed quorum response can therefore leave a local value on the coordinator.
4. **Heartbeat is not replication proof.** A recent heartbeat proves that the leaseholder was reachable, not that a follower has received every write. The staleness check is an approximation based on heartbeat age.
5. **Formal linearizability is not guaranteed.** The system has no consensus protocol, durable log, lease fencing, or strict write-leader enforcement. The report therefore describes this path as linearizable-style / majority-based rather than claiming a production-grade linearizability guarantee.
6. **Simplified AP behaviour.** ALWAYS_AP accepts at the receiving node; it does not implement full Dynamo-style sloppy quorum, hinted handoff, vector clocks, or concurrent-version preservation.
7. **LWW assumptions.** LWW depends on meaningful timestamps. Clock skew between machines could select an incorrect winner in a real distributed deployment.
8. **Union merge.** Union Merge is discussed conceptually but is not implemented; the tested conflict policy is LWW.

These limitations do not invalidate the lab experiments, but they define their scope. The implementation is suitable for demonstrating core concepts and test scenarios, not for production use as a strongly consistent distributed database.

7. Conclusion

The laboratory extended a Python/gRPC key-value store into a three-replica system with configurable read and write strategies. The implementation demonstrated the difference between CP and AP behaviour during a simulated network partition. CP writes required two acknowledgements and failed on the one-node minority side, while AP writes were accepted by the isolated replica. After the partition healed, timestamp-based Last-Write-Wins selected the newer value and read repair propagated it to the replicas participating in the read.

Quorum arithmetic was verified for $N = 3$, $R = 2$, and $W = 2$: because $R + W = 4 > 3$, read and write quorums intersect. The automated suite completed 10 of 10 tests successfully. In the healthy-cluster benchmark, follower-stale reads were faster than leaseholder-based reads: median latency decreased from 4.31 ms to 1.74 ms, while measured availability remained 100.0% for both modes.

Overall, the results illustrate that consistency, availability, and latency are design trade-offs. Stronger coordination can increase the work and latency required for a read, while local follower reads improve response time at the cost of potentially returning older data. The project provides a practical demonstration of CAP and PACELC concepts and also shows why production systems require stronger mechanisms for durability, leader authority, clock handling, and failure recovery.

Appendix A. Reproduction Commands

```
# Activate the virtual environment (Windows CMD)
.venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Generate gRPC Python files after editing the proto
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. kvstore.proto

# Run the automated CAP tests
python -m pytest tests/test_cap.py -v

# Run 1,000 reads per benchmark scenario python
perf_cap_bench.py

# Optional additional leaseholder-outage experiment python
perf_cap_bench.py --outage
```

Appendix B. Main Project Files

File	Description
kvstore.proto	Protocol Buffers messages, enums, and gRPC service definitions.
server.py	Replica implementation: writes, reads, replication, heartbeats, and read repair.
client.py	Command-line client and reusable KVClient wrapper.
kvstore_pb2.py / kvstore_pb2_grpc.py	Generated Python code for Protocol Buffers and gRPC.
tests/test_cap.py	Ten automated tests for CP/AP, partition recovery, quorum, staleness, and idempotency.
perf_cap_bench.py	Latency and availability benchmark for the two read modes.
logs/perf_cap_results.md	Saved benchmark output; the report uses the latest console results supplied for this submission.